

THE DATABASE IS A DETAIL



From an architectural point of view, the database is a non-entity—it is a detail that does not rise to the level of an architectural element. Its relationship to the architecture of a software system is rather like the relationship of a doorknob to the architecture of your home.

I realize that these are fighting words. Believe me, I've had the fight. So let me be clear: I am not talking about the data model. The structure you give to the data within your application is highly significant to the architecture of your system. But the database is not the data model. The database is piece of software. The database is a utility that provides access to the data. From the architecture's point of view, that utility is irrelevant because it's a low-level detail—a mechanism. And a good architect does not allow low-level mechanisms to pollute the system architecture.

RELATIONAL DATABASES

Edgar Codd defined the principles of relational databases in 1970. By the mid-1980s, the relational model had grown to become the dominant form of data storage. There was a good reason for this popularity: The relational model is elegant, disciplined, and robust. It is an excellent data storage and access technology.

But no matter how brilliant, useful, and mathematically sound a technology it is, it is still just a technology. And that means it's a detail.

While relational tables may be convenient for certain forms of data access, there is nothing architecturally significant about arranging data into rows within tables. The use cases of your application should neither know nor care about such matters. Indeed, knowledge of the tabular structure of the data should be restricted to the lowest-level utility functions in the outer circles of the architecture.

Many data access frameworks allow database rows and tables to be passed around the system as objects. Allowing this is an architectural error. It couples the use cases, business rules, and in some cases even the UI to the relational structure of the data.

WHY ARE DATABASE SYSTEMS SO PREVALENT?

Why are software systems and software enterprises dominated by database systems? What accounts for the preeminence of Oracle, MySQL, and SQL Server? In a word: disks.

The rotating magnetic disk was the mainstay of data storage for five decades. Several generations of programmers have known no other form of data storage. Disk technology has grown from huge stacks of massive platters 48 inches in diameter that weighed thousands of pounds and held 20 megabytes, to single thin circles, 3 inches in diameter, that weigh just a few grams and hold a terabyte or more. *It's been a wild ride.* And throughout that ride programmers have been plagued by one fatal trait of disk technology: Disks are *slow*.

On a disk, data is stored within circular tracks. Those tracks are divided into sectors that hold a convenient number of bytes, often 4K. Each platter may have hundreds of tracks, and there can be a dozen or so platters. If you want to read a particular byte

off the disk, you have to move the head to the proper track, wait for the disk to rotate to the proper sector, read all 4K of that sector into RAM, and then index into that RAM buffer to get the byte you want. And all that takes time—milliseconds of times.

Milliseconds might not seem like a lot, but a millisecond is a million times longer than the cycle time of most processors. If that data was not on a disk, it could be accessed in nanoseconds, instead of milliseconds.

To mitigate the time delay imposed by disks, you need indexes, caches, and optimized query schemes; and you need some kind of regular means of representing the data so that these indexes, caches, and query schemes know what they are working with. In short, you need a data access and management system. Over the years these systems have split into two distinct kinds: file systems and relational database management systems (RDBMS).

File systems are document based. They provide a natural and convenient way to store whole documents. They work well when you need to save and retrieve a set of documents by name, but they don't offer a lot of help when you're searching the content of those documents. It's easy to find a file named `login.c`, but it's hard, and slow, to find every `.c` file that has a variable named `x` in it.

Database systems are content based. They provide a natural and convenient way to find records based on their content. They are very good at associating multiple records based on some bit of content that they all share. Unfortunately, they are rather poor at storing and retrieving opaque documents.

Both of these systems organize the data on disk so that it can be stored and retrieved in as efficient a way as possible, given their particular access needs. Each has their own scheme for indexing and arranging the data. In addition, each eventually brings the relevant data into RAM, where it can be quickly manipulated.

WHAT IF THERE WERE NO DISK?

As prevalent as disks once were, they are now a dying breed. Soon they will have gone the way of tape drives, floppy drives, and CDs. They are being replaced by RAM.

Ask yourself this question: When all the disks are gone, and all your data is stored in

RAM, how will you organize that data? Will you organize it into tables and access it with SQL? Will you organize it into files and access it through a directory?

Of course not. You'll organize it into linked lists, trees, hash tables, stacks, queues, or any of the other myriad data structures, and you'll access it using pointers or references—because *that's what programmers do*.

In fact, if you think carefully about this issue, you'll realize that this is what you already do. Even though the data is kept in a database or a file system, you read it into RAM and then you reorganize it, for your own convenience, into lists, sets, stacks, queues, trees, or whatever data structure meets your fancy. It is very unlikely that you leave the data in the form of files or tables.

DETAILS

This reality is why I say that the database is a detail. It's just a mechanism we use to move the data back and forth between the surface of the disk and RAM. The database is really nothing more than a big bucket of bits where we store our data on a long-term basis. But we seldom use the data in that form.

Thus, from an architectural viewpoint, we should not care about the form that the data takes while it is on the surface of a rotating magnetic disk. Indeed, we should not acknowledge that the disk exists at all.

BUT WHAT ABOUT PERFORMANCE?

Isn't performance an architectural concern? Of course it is—but when it comes to data storage, it's a concern that can be entirely encapsulated and separated from the business rules. Yes, we need to get the data in and out of the data store quickly, but that's a low-level concern. We can address that concern with low-level data access mechanisms. It has nothing whatsoever to do with the overall architecture of our systems.

ANECDOTE

In the late 1980s, I led a team of software engineers at a startup company that was trying to build and market a network management system that measured the

communications integrity of T1 telecommunication lines. The system retrieved data from the devices at the endpoints of those lines, and then ran a series of predictive algorithms to detect and report problems.

We were using UNIX platforms, and we stored our data in simple random access files. We had no need of a relational database because our data had few content-based relationships. It was better kept in trees and linked lists in those random access files. In short, we kept the data in a form that was most convenient to load into RAM where it could be manipulated.

We hired a marketing manager for this startup—a nice and knowledgeable guy. But he immediately told me that we had to have a relational database in the system. It wasn't an option and it wasn't an engineering issue—it was a marketing issue.

This made no sense to me. Why in the world would I want to rearrange my linked lists and trees into a bunch of rows and tables accessed through SQL? Why would I introduce all the overhead and expense of a massive RDBMS when a simple random access file system was more than sufficient? So I fought him, tooth and nail.

We had a hardware engineer at this company who took up the RDBMS chant. He became convinced that our software system needed an RDBMS for technical reasons. He held meetings behind my back with the executives of the company, drawing stick figures on the whiteboard of a house balancing on a pole, and he would ask the executives, “Would you build a house on a pole?” His implied message was that an RDBMS that keeps its tables in random access files was somehow more reliable than the random access files that we were using.

I fought him. I fought the marketing guy. I stuck to my engineering principles in the face of incredible ignorance. I fought, and fought, and fought.

In the end, the hardware developer was promoted over my head to become the software manager. In the end, they put a RDBMS into that poor system. And, in the end, they were absolutely right and I was wrong.

Not for engineering reasons, mind you: I was right about that. I was right to fight against putting an RDBMS into the architectural core of the system. The reason I was wrong was because our customers expected us to have a relational database. They didn't know what they would do with it. They didn't have any realistic way of using the relational data in our system. But it didn't matter: Our customers fully expected an RDBMS. It had become a check box item that all the software purchasers had on their list. There was no engineering rationale—rationality had

nothing to do with it. It was an irrational, external, and entirely baseless need, but it was no less real.

Where did that need come from? It originated from the highly effective marketing campaigns employed by the database vendors at the time. They had managed to convince high-level executives that their corporate “data assets” needed protection, and that the database systems they offered were the ideal means of providing that protection.

We see the same kind of marketing campaigns today. The word “enterprise” and the notion of “Service-Oriented Architecture” have much more to do with marketing than with reality.

What *should* I have done in that long-ago scenario? I should have bolted an RDBMS on the side of the system and provided some narrow and safe data access channel to it, while maintaining the random access files in the core of the system. What *did* I do? I quit and became a consultant.

CONCLUSION

The organizational structure of data, the data model, is architecturally significant. The technologies and systems that move data on and off a rotating magnetic surface are not. Relational database systems that force the data to be organized into tables and accessed with SQL have much more to do with the latter than with the former. The data is significant. The database is a detail.